



Natural Language Processing with Convolutional Neural Networks and Transformers

2019–20

Ion Androutsopoulos

<http://www.aueb.gr/users/ion/>

Contents

- Convolutional neural networks (CNNs) and applications in NLP.
- Image to text generation with CNN encoders and RNN decoders.
- Key-value attention, multi-head attention, Transformers, BERT.

Convolutions on images

Averaging each pixel with its neighboring values blurs an image:

0	0	0	0	0
0	1	1	1	0
0	1	1	1	0
0	1	1	1	0
0	0	0	0	0

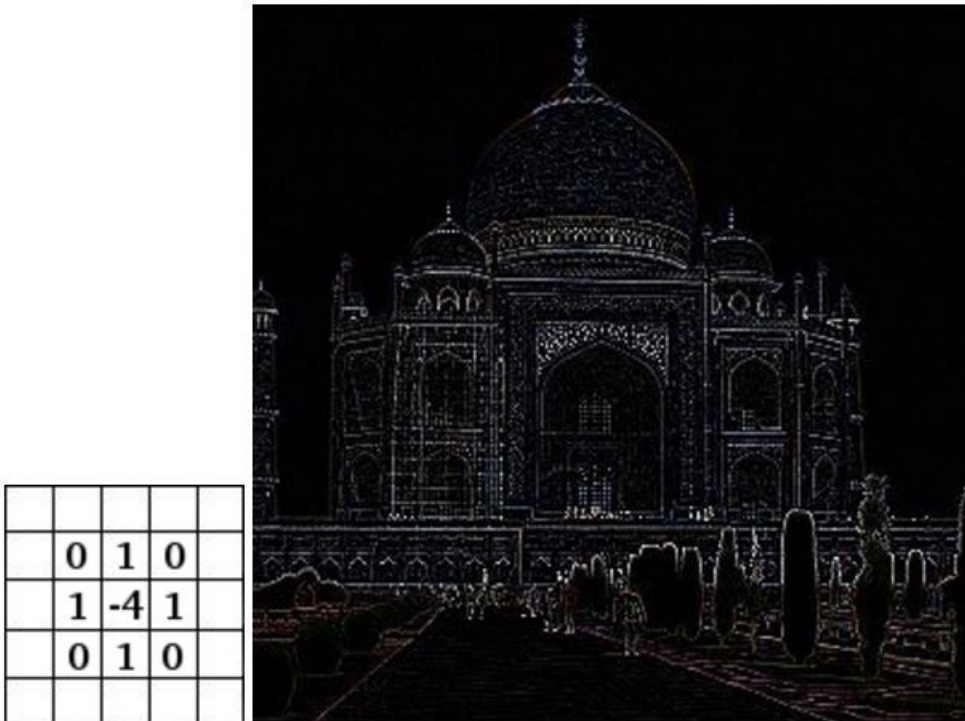


From the blog post
“Understanding
Convolutional Neural
Networks for NLP” of
Denny Britz, 2015.
[http://www.wildml.com/
2015/11/understanding-
convolutional-neural-
networks-for-nlp/](http://www.wildml.com/2015/11/understanding-convolutional-neural-networks-for-nlp/)

Convolutions on images

Taking the difference between a pixel and its neighbors detects edges:

(To understand this one intuitively, think about what happens in parts of the image that are smooth, where a pixel color equals that of its neighbors: The additions cancel and the resulting value is 0, or black. If there's a sharp edge in intensity, a transition from white to black for example, you get a large difference and a resulting white value)



From the blog post
“Understanding
Convolutional Neural
Networks for NLP” of
Denny Britz, 2015.
[http://www.wildml.com/
2015/11/understanding-
convolutional-neural-
networks-for-nlp/](http://www.wildml.com/2015/11/understanding-convolutional-neural-networks-for-nlp/)

Convolutions on text

Let's **pretend** that we know what the **dimensions** of the word **embeddings represent**, and that the dimensions are **binary**.

2

Words	Embeddings			
	Subject	Positive	Stress	Quantity
I	1	0	0	0
like	0	1	0	0
this	0	0	0	0
movie	0	0	0	0
very	0	0	1	0
much	0	0	0	1
!	0	0	1	0

Filter for “I like”, “we admire”...			
1	0	0	0
0	1	0	0

Filter for “very much”, “so much”...			
0	0	1	0
0	0	0	1

Convolutions on text

Let's **pretend** that we know what the **dimensions** of the word **embeddings represent**, and that the dimensions are **binary**.

Words	Embeddings			
	Subject	Positive	Stress	Quantity
I	1	0	0	0
like	0	1	0	0
this	0	0	0	0
movie	0	0	0	0
very	0	0	1	0
much	0	0	0	1
!	0	0	1	0

0

Filter for “I like”, “we admire”...			
1	0	0	0
0	1	0	0

Filter for “very much”, “so much”...			
0	0	1	0
0	0	0	1

Convolutions on text

Words	Embeddings			
	Subject	Positive	Stress	Quantity
I	1	0	0	0
like	0	1	0	0
this	0	0	0	0
movie	0	0	0	0
very	0	0	1	0
much	0	0	0	1
!	0	0	1	0

2
0
0
0
0
0
0

Filter for “I like”, “we admire”...			
1	0	0	0
0	1	0	0

Filter for “very much”, “so much”...			
0	0	1	0
0	0	0	1

Convolutions on text

Words	Embeddings			
	Subject	Positive	Stress	Quantity
I	1	0	0	0
like	0	1	0	0
this	0	0	0	0
movie	0	0	0	0
very	0	0	1	0
much	0	0	0	1
!	0	0	1	0

2 0
0 0
0 0
0 0
0
0

Filter for “I like”, “we admire”...			
1	0	0	0
0	1	0	0

Filter for “very much”, “so much”...			
0	0	1	0
0	0	0	1

Convolutions on text

Words	Embeddings					
	Subject	Positive	Stress	Quantity		
I	1	0	0	0		
like	0	1	0	0	2	0
this	0	0	0	0	0	0
movie	0	0	0	0	0	0
very	0	0	1	0	0	0
much	0	0	0	1	0	2
!	0	0	1	0	0	

Filter for “I like”, “we admire”...			
1	0	0	0
0	1	0	0

Filter for “very much”, “so much”...			
0	0	1	0
0	0	0	1

Convolutions on text

Words	Embeddings			
	Subject	Positive	Stress	Quantity
I	1	0	0	0
like	0	1	0	0
this	0	0	0	0
movie	0	0	0	0
very	0	0	1	0
much	0	0	0	1
!	0	0	1	0

2 0
0 0
0 0
0 0
0 2
0 0

Best scores of the two filters:
to what extent they **match**
anywhere in the sentence.



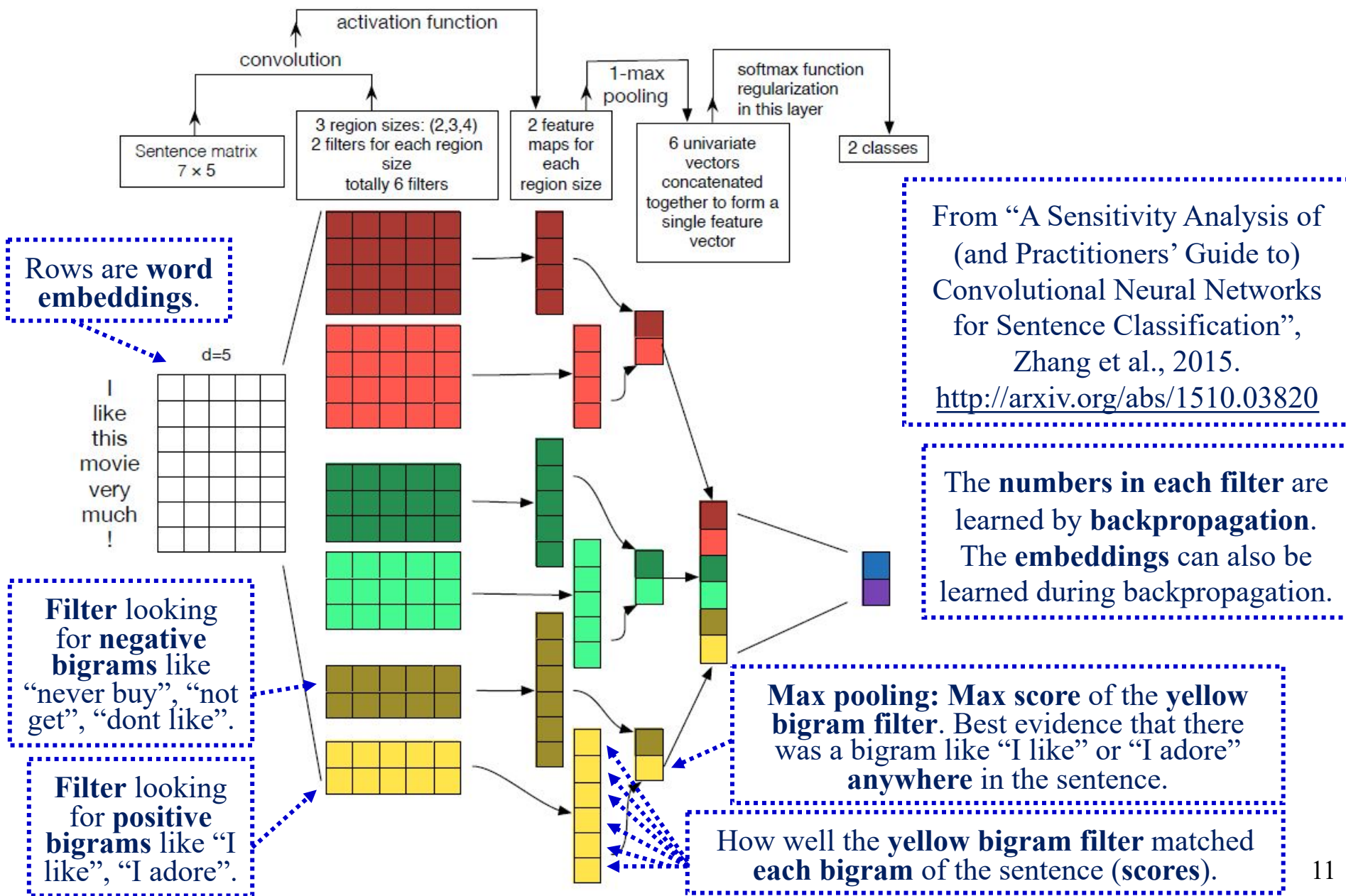
max
pooling

2 2

Filter for “I like”, “we admire”...			
1	0	0	0
0	1	0	0

Filter for “very much”, “so much”...			
0	0	1	0
0	0	0	1

Convolutional Neural Networks



Convolutions on text – closer to reality

Words	Embeddings			
	d_1	d_2	d_3	d_4
I	$x_{1,1}$	$x_{1,2}$	$x_{1,3}$	$x_{1,4}$
like	$x_{2,1}$	$x_{2,2}$	$x_{2,3}$	$x_{2,4}$
this	$x_{3,1}$	$x_{3,2}$	$x_{3,3}$	$x_{3,4}$
movie	$x_{4,1}$	$x_{4,2}$	$x_{4,3}$	$x_{4,4}$
very	$x_{5,1}$	$x_{5,2}$	$x_{5,3}$	$x_{5,4}$
much	$x_{6,1}$	$x_{6,2}$	$x_{6,3}$	$x_{6,4}$
!	$x_{7,1}$	$x_{7,2}$	$x_{7,3}$	$x_{7,4}$

$$\text{ReLU}(wx + b)$$

$$x^T = \langle x_{1,1}, x_{1,2}, x_{1,3}, \dots, x_{2,3}, x_{2,4} \rangle$$

A bigram filter			
$w_{1,1}$	$w_{1,2}$	$w_{1,3}$	$w_{1,4}$
$w_{2,1}$	$w_{2,2}$	$w_{2,3}$	$w_{2,4}$

$$w = \langle w_{1,1}, w_{1,2}, w_{1,3}, \dots, w_{2,3}, w_{2,4} \rangle$$

$$b$$

Convolutions on text – closer to reality

Words	Embeddings			
	d_1	d_2	d_3	d_4
I	$x_{1,1}$	$x_{1,2}$	$x_{1,3}$	$x_{1,4}$
like	$x_{2,1}$	$x_{2,2}$	$x_{2,3}$	$x_{2,4}$
this	$x_{3,1}$	$x_{3,2}$	$x_{3,3}$	$x_{3,4}$
movie	$x_{4,1}$	$x_{4,2}$	$x_{4,3}$	$x_{4,4}$
very	$x_{5,1}$	$x_{5,2}$	$x_{5,3}$	$x_{5,4}$
much	$x_{6,1}$	$x_{6,2}$	$x_{6,3}$	$x_{6,4}$
!	$x_{7,1}$	$x_{7,2}$	$x_{7,3}$	$x_{7,4}$

$$\text{ReLU}(wx + b)$$

$$x^T = \langle x_{2,1}, x_{2,2}, x_{2,3}, \dots, x_{3,3}, x_{3,4} \rangle$$

A bigram filter			
$w_{1,1}$	$w_{1,2}$	$w_{1,3}$	$w_{1,4}$
$w_{2,1}$	$w_{2,2}$	$w_{2,3}$	$w_{2,4}$

$$w = \langle w_{1,1}, w_{1,2}, w_{1,3}, \dots, w_{2,3}, w_{2,4} \rangle$$

$$b$$

Now applying **three** bigram filters

Words	Embeddings			
	d_1	d_2	d_3	d_4
I	$x_{1,1}$	$x_{1,2}$	$x_{1,3}$	$x_{1,4}$
like	$x_{2,1}$	$x_{2,2}$	$x_{2,3}$	$x_{2,4}$
this	$x_{3,1}$	$x_{3,2}$	$x_{3,3}$	$x_{3,4}$
movie	$x_{4,1}$	$x_{4,2}$	$x_{4,3}$	$x_{4,4}$
very	$x_{5,1}$	$x_{5,2}$	$x_{5,3}$	$x_{5,4}$
much	$x_{6,1}$	$x_{6,2}$	$x_{6,3}$	$x_{6,4}$
!	$x_{7,1}$	$x_{7,2}$	$x_{7,3}$	$x_{7,4}$

$$h_2 = \text{ReLU}(Wx + b) \in \mathbb{R}^{3 \times 1}$$

$$x^T = \langle x_{2,1}, x_{2,2}, \dots, x_{3,3}, x_{3,4} \rangle \in \mathbb{R}^{1 \times 8}$$

$$W = \begin{bmatrix} w_{1,1,1} & w_{1,1,2} & w_{1,1,3} & \dots & w_{1,2,3} & w_{1,2,4} \\ w_{2,1,1} & w_{2,1,2} & w_{2,1,3} & \dots & w_{2,2,3} & w_{2,2,4} \\ w_{3,1,1} & w_{3,1,2} & w_{3,1,3} & \dots & w_{3,2,3} & w_{3,2,4} \end{bmatrix} \in \mathbb{R}^{3 \times 8} \quad b = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} \in \mathbb{R}^{3 \times 1}$$

Applying 3 bigram filters

Words	Embeddings			
	d_1	d_2	d_3	d_4
I	$x_{1,1}$	$x_{1,2}$	$x_{1,3}$	$x_{1,4}$
like	$x_{2,1}$	$x_{2,2}$	$x_{2,3}$	$x_{2,4}$
this	$x_{3,1}$	$x_{3,2}$	$x_{3,3}$	$x_{3,4}$
movie	$x_{4,1}$	$x_{4,2}$	$x_{4,3}$	$x_{4,4}$
very	$x_{5,1}$	$x_{5,2}$	$x_{5,3}$	$x_{5,4}$
much	$x_{6,1}$	$x_{6,2}$	$x_{6,3}$	$x_{6,4}$
!	$x_{7,1}$	$x_{7,2}$	$x_{7,3}$	$x_{7,4}$

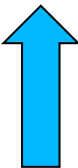
$$h_2 = \langle \textcolor{red}{h}_{2,1}, \textcolor{green}{h}_{2,2}, \textcolor{violet}{h}_{2,3} \rangle^T \in \mathbb{R}^{3 \times 1}$$

$$W = \begin{bmatrix} \textcolor{red}{w}_{1,1,1} & \textcolor{red}{w}_{1,1,2} & \textcolor{red}{w}_{1,1,3} & \cdots & \textcolor{red}{w}_{1,2,3} & \textcolor{red}{w}_{1,2,4} \\ \textcolor{green}{w}_{2,1,1} & \textcolor{green}{w}_{2,1,2} & \textcolor{green}{w}_{2,1,3} & \cdots & \textcolor{green}{w}_{2,2,3} & \textcolor{green}{w}_{2,2,4} \\ \textcolor{violet}{w}_{3,1,1} & \textcolor{violet}{w}_{3,1,2} & \textcolor{violet}{w}_{3,1,3} & \cdots & \textcolor{violet}{w}_{3,2,3} & \textcolor{violet}{w}_{3,2,4} \end{bmatrix} \in \mathbb{R}^{3 \times 8} \quad b = \begin{bmatrix} \textcolor{red}{b}_1 \\ \textcolor{green}{b}_2 \\ \textcolor{violet}{b}_3 \end{bmatrix} \in \mathbb{R}^{3 \times 1}$$

Applying 3 bigram filters

$$h^{max} = \langle \max(h_{*,1}), \max(h_{*,2}), \max(h_{*,3}) \rangle^T$$

max
pooling



Feature vector
sent to a
classifier,
regressor, etc.

Words

I
like
this
movie
very
much
!

Embeddings			
d_1	d_2	d_3	d_4
$x_{1,1}$	$x_{1,2}$	$x_{1,3}$	$x_{1,4}$
$x_{2,1}$	$x_{2,2}$	$x_{2,3}$	$x_{2,4}$
$x_{3,1}$	$x_{3,2}$	$x_{3,3}$	$x_{3,4}$
$x_{4,1}$	$x_{4,2}$	$x_{4,3}$	$x_{4,4}$
$x_{5,1}$	$x_{5,2}$	$x_{5,3}$	$x_{5,4}$
$x_{6,1}$	$x_{6,2}$	$x_{6,3}$	$x_{6,4}$
$x_{7,1}$	$x_{7,2}$	$x_{7,3}$	$x_{7,4}$

$$h_1 = \langle h_{1,1}, h_{1,2}, h_{1,3} \rangle^T$$

$$h_2 = \langle h_{2,1}, h_{2,2}, h_{2,3} \rangle^T$$

$$h_3 = \langle h_{3,1}, h_{3,2}, h_{3,3} \rangle^T$$

$$h_4 = \langle h_{4,1}, h_{4,2}, h_{4,3} \rangle^T$$

...

$$h_7 = \langle h_{7,1}, h_{7,2}, h_{7,3} \rangle^T$$

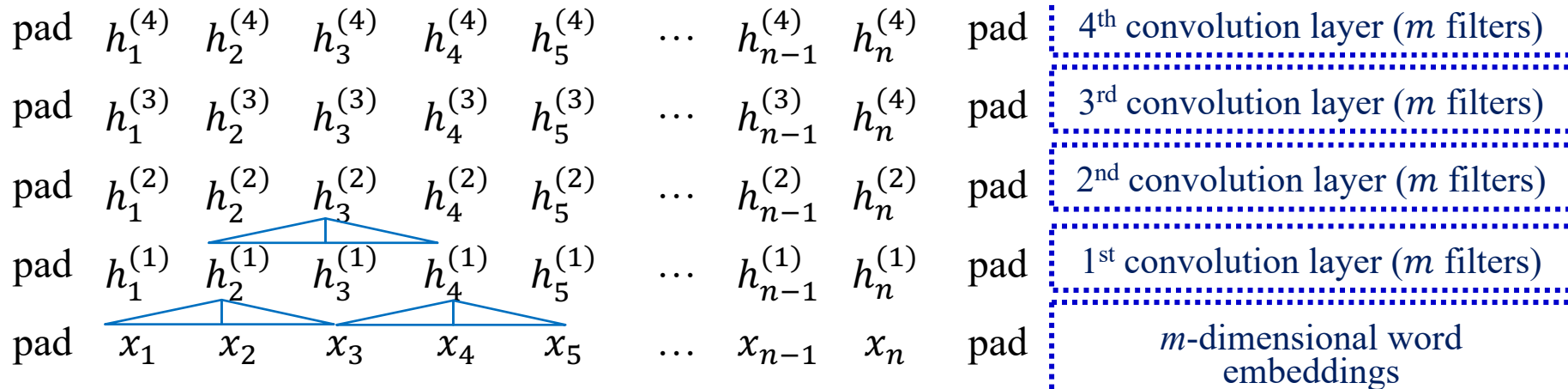
$$W = \begin{bmatrix} w_{1,1,1} & w_{1,1,2} & w_{1,1,3} & \dots & w_{1,2,3} & w_{1,2,4} \\ w_{2,1,1} & w_{2,1,2} & w_{2,1,3} & \dots & w_{2,2,3} & w_{2,2,4} \\ w_{3,1,1} & w_{3,1,2} & w_{3,1,3} & \dots & w_{3,2,3} & w_{3,2,4} \end{bmatrix} \in \mathbb{R}^{3 \times 8} \quad b = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} \in \mathbb{R}^{3 \times 1}$$

Stacked CNNs for classification/regression

$$h^{max} = \left\langle \text{max}\left(h_{*,1}^{(4)}\right), \text{max}\left(h_{*,2}^{(4)}\right), \dots, \text{max}\left(h_{*,m}^{(4)}\right) \right\rangle^T \in \mathbb{R}^{1 \times m}$$

max
pooling

Feature vector sent to a document classifier or regressor (e.g., MLP).

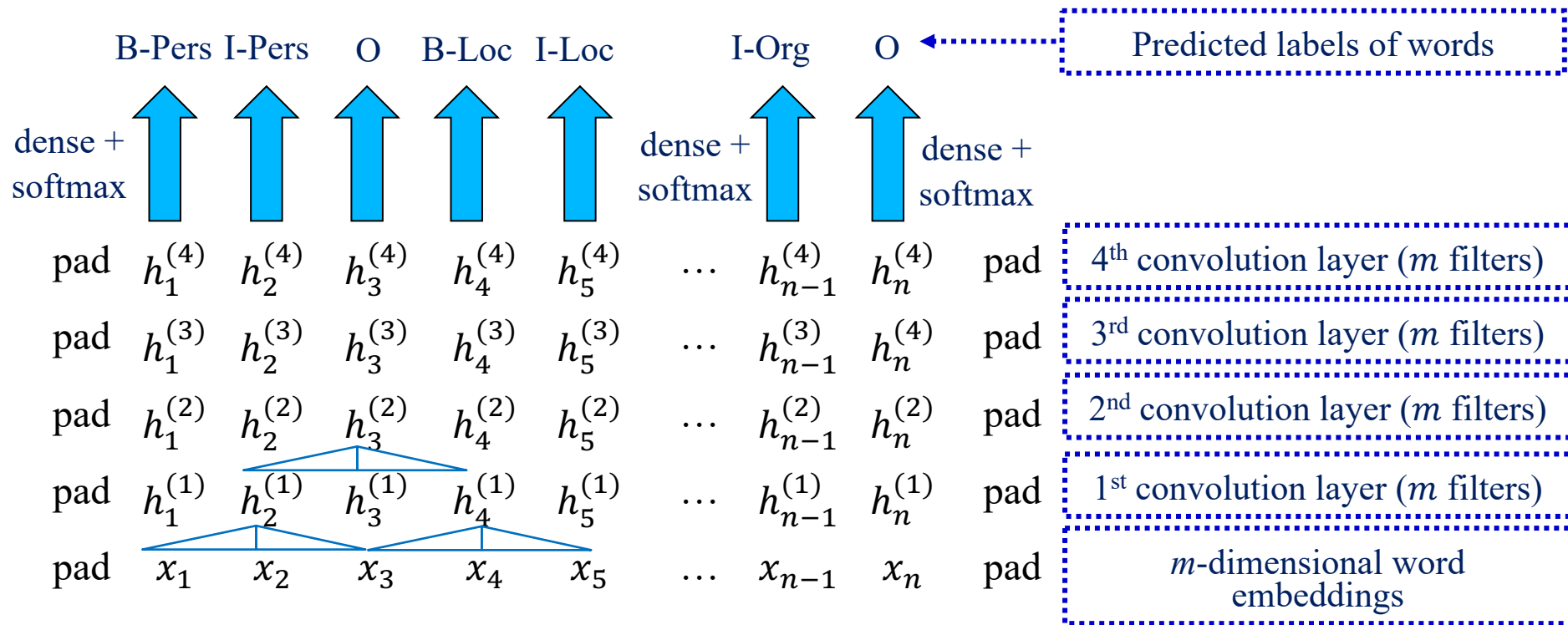


$$h_i^{(1)} = \text{ReLU}\left(W^{(1)}[x_{i-1}; x_i; x_{i+1}] + b^{(1)}\right) + x_i \in \mathbb{R}^{m \times 1}$$

$$h_i^{(j)} = \text{ReLU}\left(W^{(j)}[h_{i-1}^{(j-1)}; h_i^{(j-1)}; h_{i+1}^{(j-1)}] + b^{(j)}\right) + h_i^{(j-1)} \in \mathbb{R}^{m \times 1}$$

Residual (shortcut) connection, needed when stacking many CNNs (or RNNs).

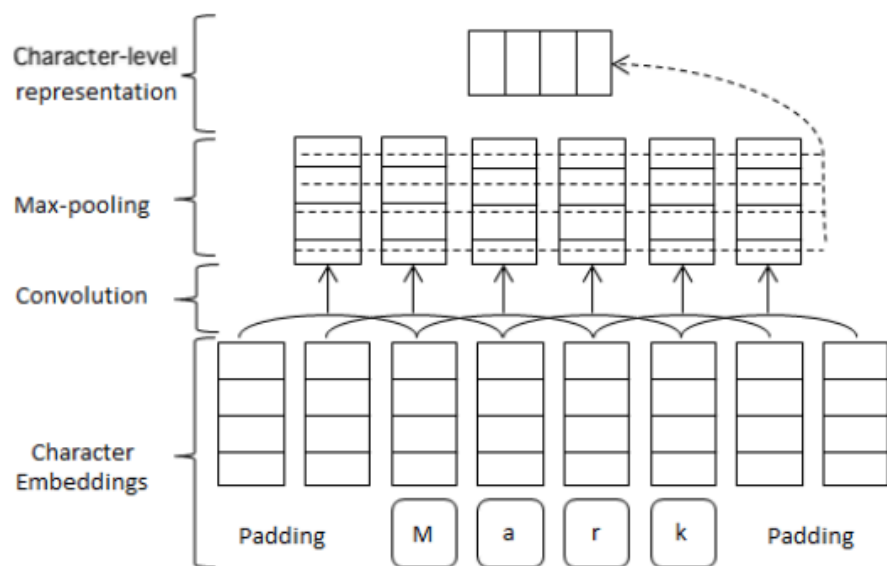
Stacked CNNs for sequence labeling



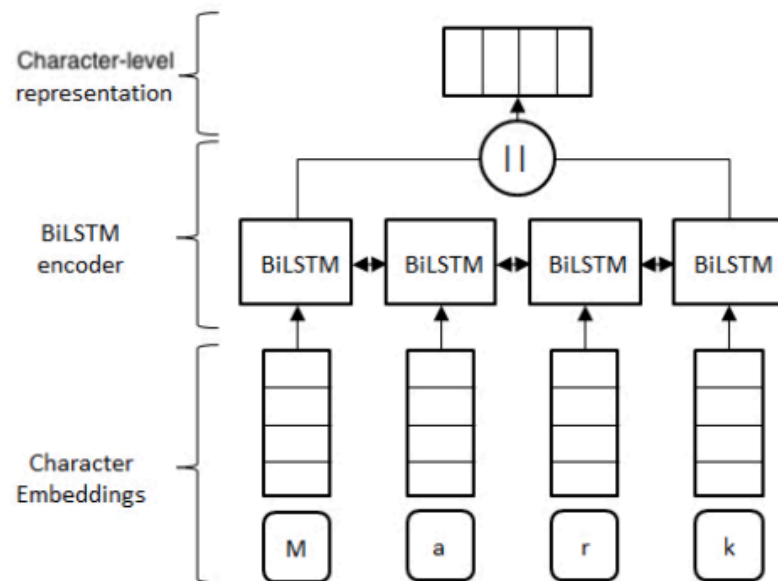
$$h_i^{(1)} = \text{ReLU}(W^{(1)}[x_{i-1}; x_i; x_{i+1}] + b^{(1)}) + x_i \in \mathbb{R}^{m \times 1}$$

$$h_i^{(j)} = \text{ReLU}(W^{(j)}[h_{i-1}^{(j-1)}; h_i^{(j-1)}; h_{i+1}^{(j-1)}] + b^{(j)}) + h_i^{(j-1)} \in \mathbb{R}^{m \times 1}$$

CNNs/RNNs that produce word embeddings from character embeddings



(CNN-based character-level word representation)



(LSTM-based character-level word representation)

Figure 2: Character-level word representations. This figure is also adapted from Reimers and Gurevych (2017a).

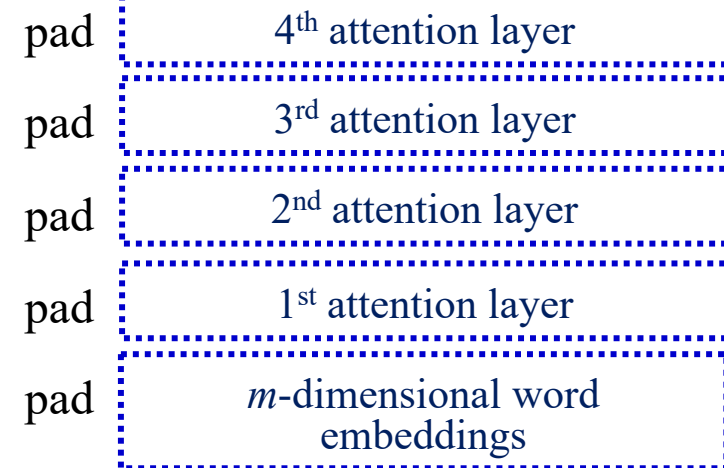
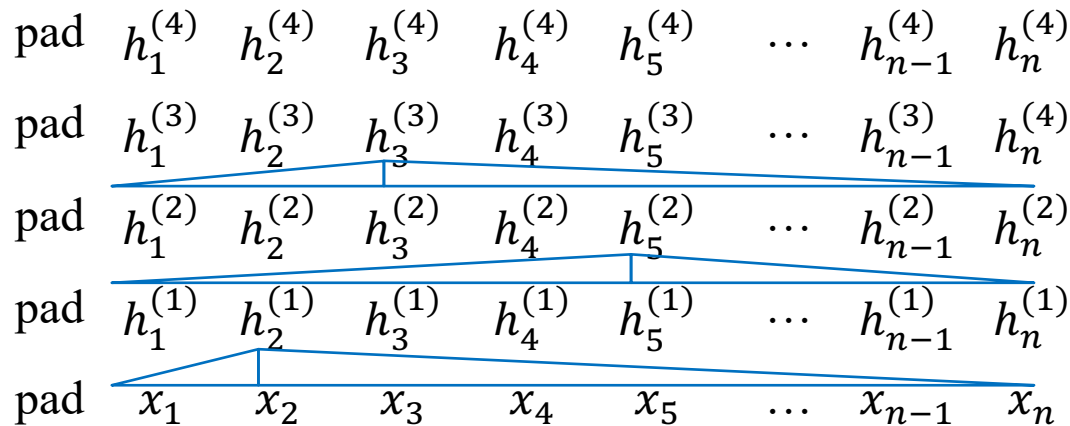
Z. Zhai, D.Q. Nguyen and K. Verspoor, “Comparing CNN and LSTM Character-Level Embeddings in BiLSTM-CRF Models for Chemical and Disease Named Entity Recognition”. 9th Int. Workshop on Health Text Mining and Information Analysis, Brussels, Belgium, 2018. <http://aclweb.org/anthology/W18-5605>

Replacing Convolutions with Attention

$$h^{max} = \left\langle \max(h_{*,1}^{(4)}), \max(h_{*,2}^{(4)}), \dots, \max(h_{*,m}^{(4)}) \right\rangle^T \in \mathbb{R}^{1 \times m}$$

max
pooling

Feature vector sent to a document classifier or regressor (e.g., MLP).

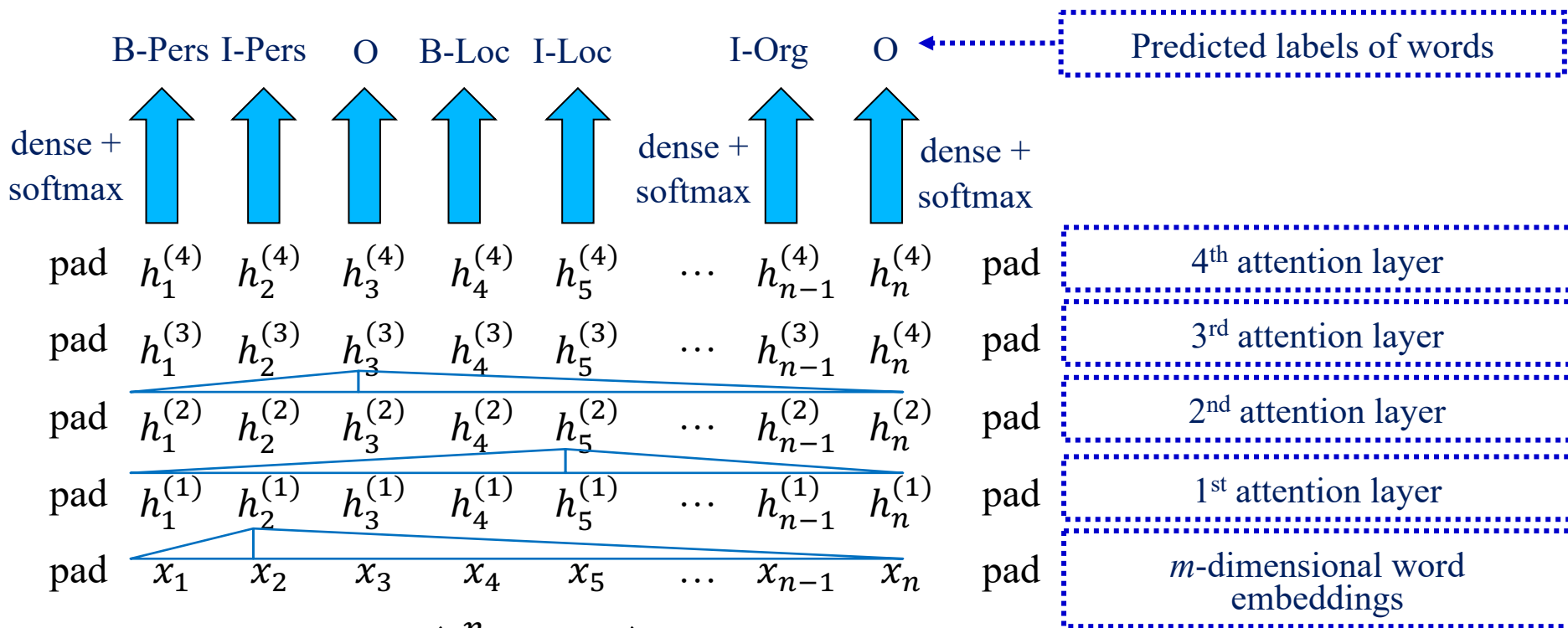


$$h_i^{(1)} = \text{MLP}^{(1)} \left(\sum_{r=1}^n a_{i,r}^{(1)} x_r \right) \in \mathbb{R}^{m \times 1}$$

$$h_i^{(j)} = \text{MLP}^{(j)} \left(\sum_{r=1}^n a_{i,r}^{(j)} h_r^{(j-1)} \right) \in \mathbb{R}^{m \times 1}$$

Without the MLP, each dimension of $h_i^{(j)}$ would only depend on the corresponding dimensions of the $h_r^{(j-1)}$ vectors.

Or for sequence labeling...

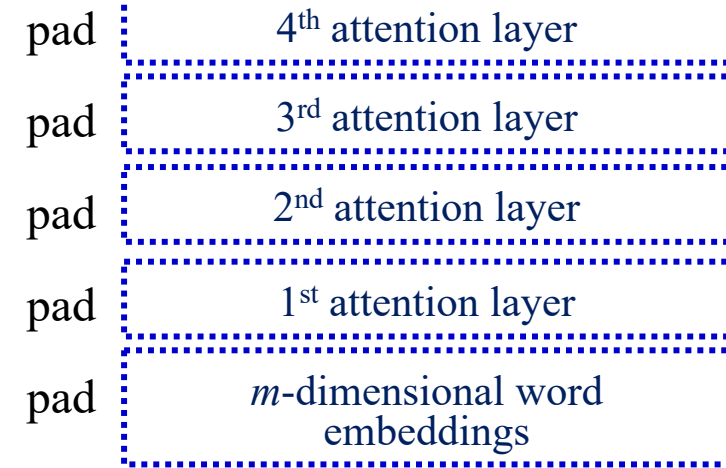
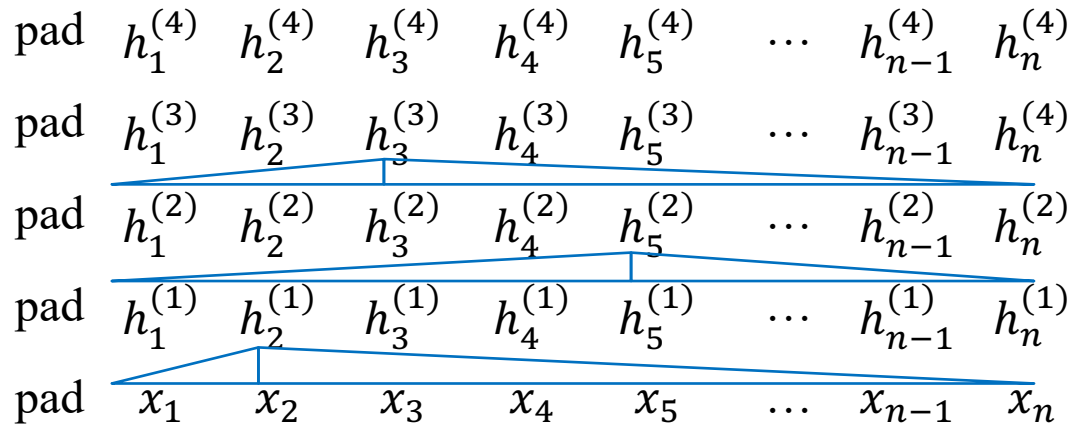


$$h_i^{(1)} = \text{MLP}^{(1)} \left(\sum_{r=1}^n a_{i,r}^{(1)} x_r \right) \in \mathbb{R}^{m \times 1}$$

$$h_i^{(j)} = \text{MLP}^{(j)} \left(\sum_{r=1}^n a_{i,r}^{(j)} h_r^{(j-1)} \right) \in \mathbb{R}^{m \times 1}$$

Without the MLP, each dimension of $h_i^{(j)}$ would only depend on the corresponding dimensions of the $h_r^{(j-1)}$ vectors.

Query-Key-Value attention



$$h_i^{(1)} = \text{MLP}^{(1)} \left(\sum_{r=1}^n a_{i,r}^{(1)} v_r^{(1)} \right) =$$

$$= \text{MLP}^{(1)} \left(\sum_{r=1}^n \text{softmax} \left(q_i^{(1)T} k_r^{(1)} \right) v_r^{(1)} \right) \in \mathbb{R}^{m \times 1}$$

$$q_i^{(1)} = W^{Q,(1)} x_i$$

$$k_r^{(1)} = W^{K,(1)} x_r$$

$$v_r^{(1)} = W^{V,(1)} x_r$$

$$h_i^{(j)} = \text{MLP}^{(j)} \left(\sum_{r=1}^n a_{i,r}^{(j)} v_r^{(j)} \right) =$$

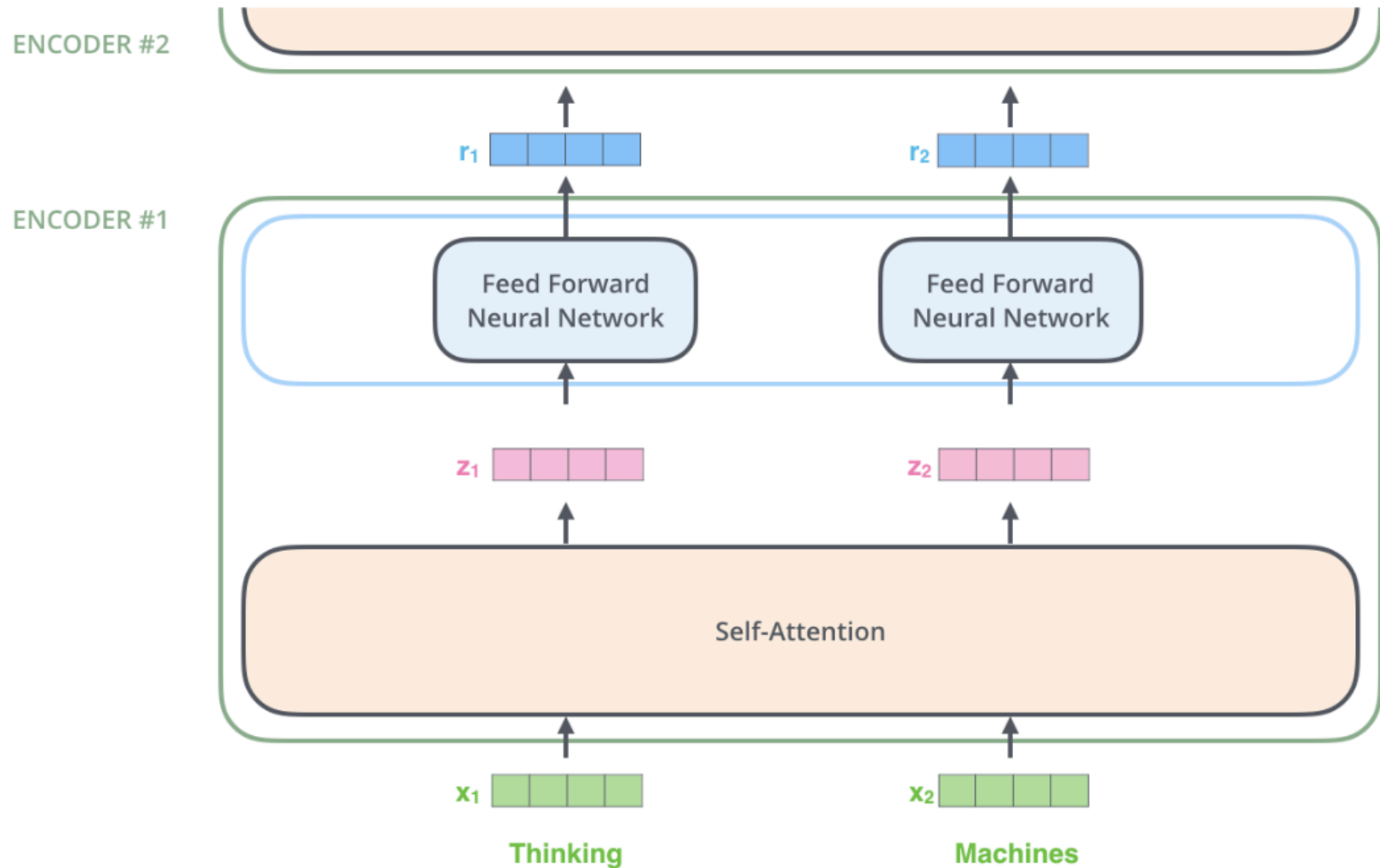
$$= \text{MLP}^{(j)} \left(\sum_{r=1}^n \text{softmax} \left(q_i^{(j)T} k_r^{(j)} \right) v_r^{(j)} \right) \in \mathbb{R}^{m \times 1}$$

$$q_i^{(j)} = W^{Q,(j)} h_i^{(j-1)}$$

$$k_r^{(j)} = W^{K,(j)} h_r^{(j-1)}$$

$$v_r^{(j)} = W^{V,(j)} h_r^{(j-1)}$$

Stacking Transformer Encoders



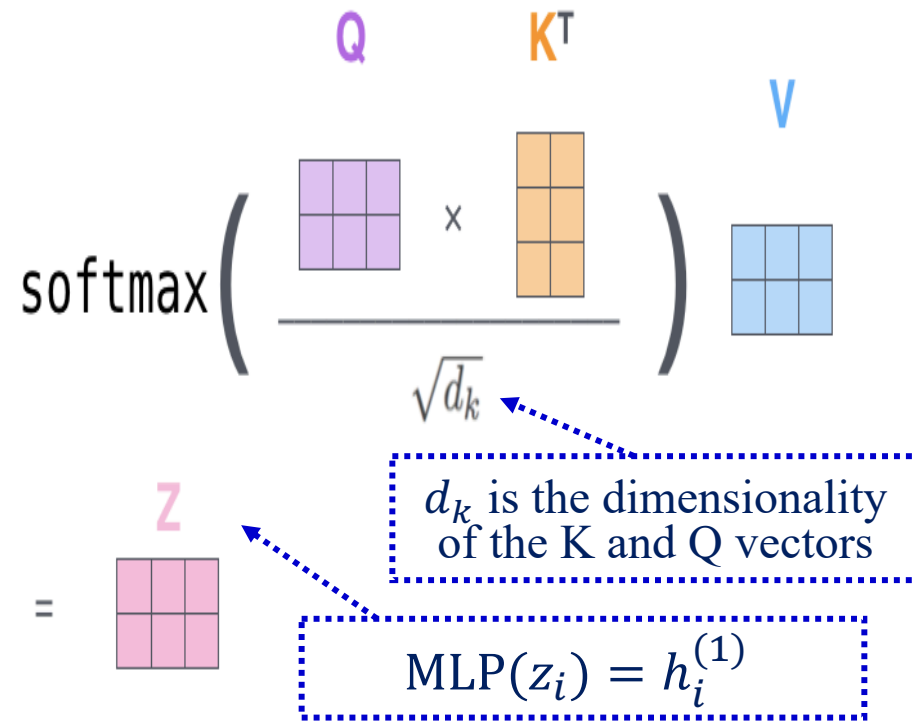
Figures from J. Alammari's "The Illustrated Transformer"
(<https://jalammar.github.io/illustrated-transformer/>). Transformers paper: Vaswani et al.,
"Attention is All You Need", 2017 (<https://arxiv.org/abs/1706.03762>).

Query-Key-Value attention via matrices

$$X \times W^Q = Q$$


$$X \times W^K = K$$


$$X \times W^V = V$$


$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) \times V = Z$$


d_k is the dimensionality of the K and Q vectors

$\text{MLP}(z_i) = h_i^{(1)}$

Figures from J. Alammam's "The Illustrated Transformer"
(<https://jalammar.github.io/illustrated-transformer/>). Transformers paper: Vaswani et al.,
"Attention is All You Need", 2017 (<https://arxiv.org/abs/1706.03762>).

Multiple attention heads

1) This is our input sentence*

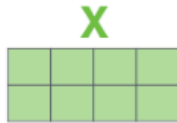
2) We embed each word*

3) Split into 8 heads. We multiply X or R with weight matrices

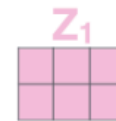
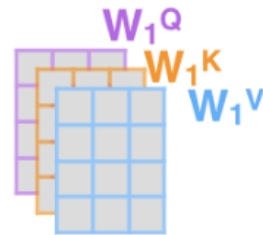
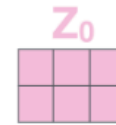
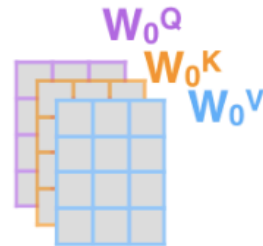
4) Calculate attention using the resulting $Q/K/V$ matrices

5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

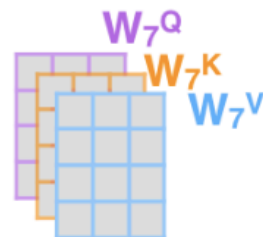
Thinking
Machines



* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



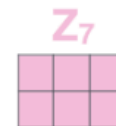
...



...



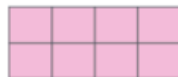
...



W^O

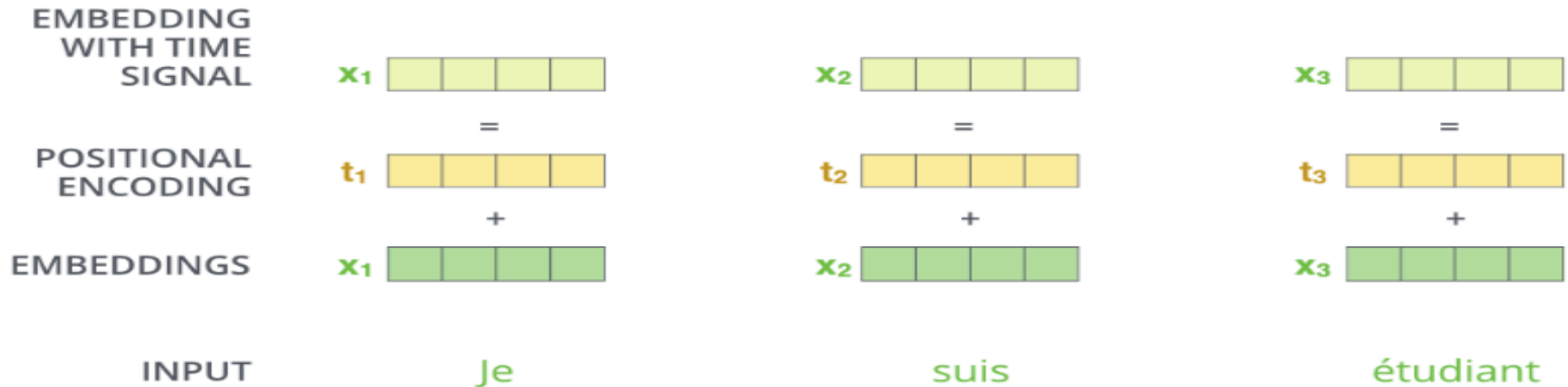


Z



Figures from J. Alammr's "The Illustrated Transformer"
(<https://jalammar.github.io/illustrated-transformer/>). Transformers paper: Vaswani et al.,
"Attention is All You Need", 2017 (<https://arxiv.org/abs/1706.03762>).

Positional encodings



Positional encodings needed to capture the **word order/positions**:

- **Without them**, Transformers are **unaware of word order**.
- **Sinusoid functions** used to produce them in the **original paper**.
- But can also be **position embeddings** learned during training.
 - Embedding of **position 1**, embedding of **position 2** etc.

Figures from J. Alammam's "The Illustrated Transformer"
(<https://jalammar.github.io/illustrated-transformer/>). Transformers paper: Vaswani et al.,
"Attention is All You Need", 2017 (<https://arxiv.org/abs/1706.03762>).

Complete Transformer encoder layer

“Feed Forward”: the **same** **MLP** at all word positions

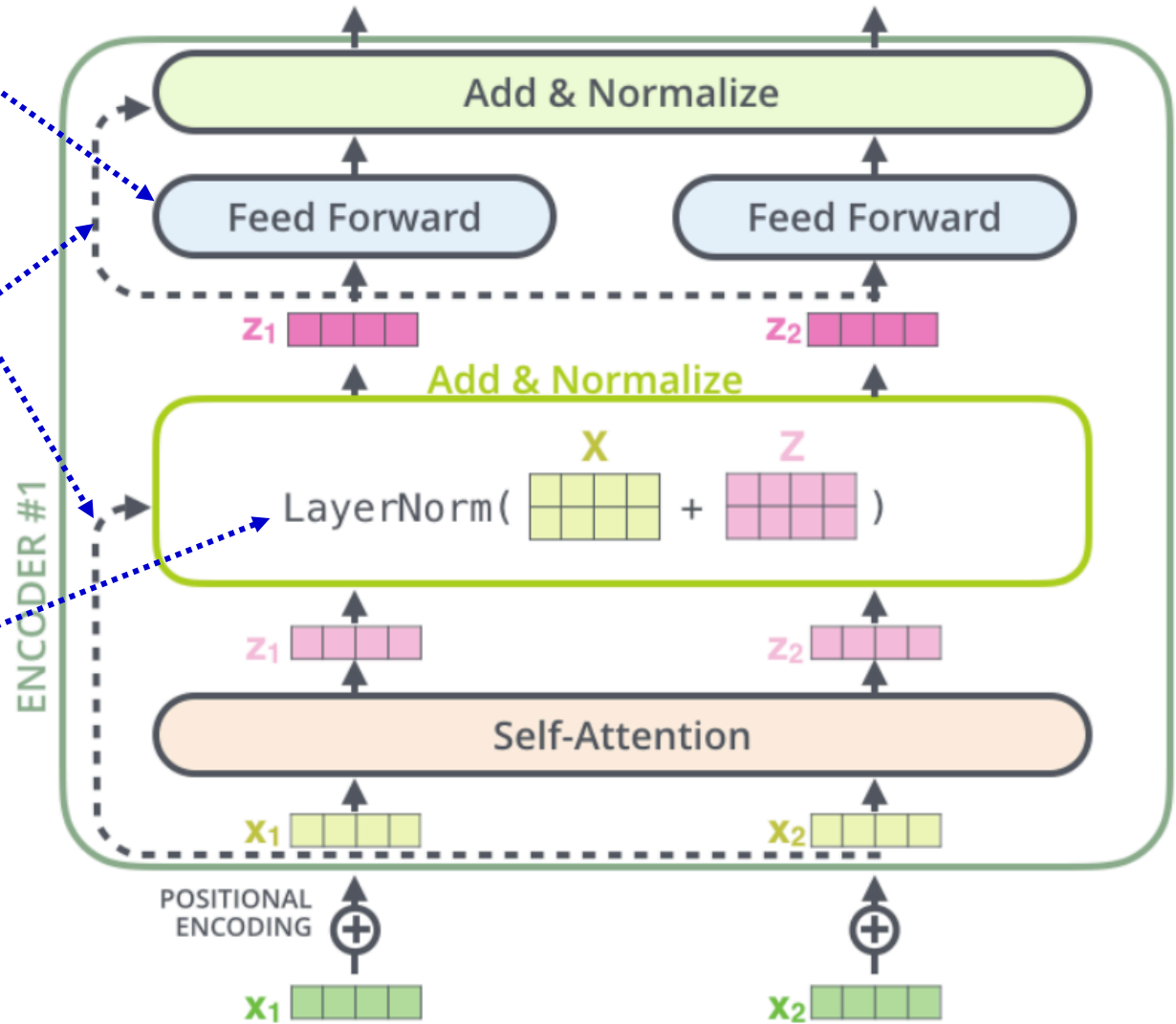
“Add”: **residual** connections

Layer Normalization:

Here, we subtract from each cell l of $(X+Z)$ the mean μ_l of its row, and divide by the std. dev σ_l , multiplying by a learned parameter g_l .

See Ba et al. “Layer Normalization”, 2016.
<https://arxiv.org/abs/1607.06450>

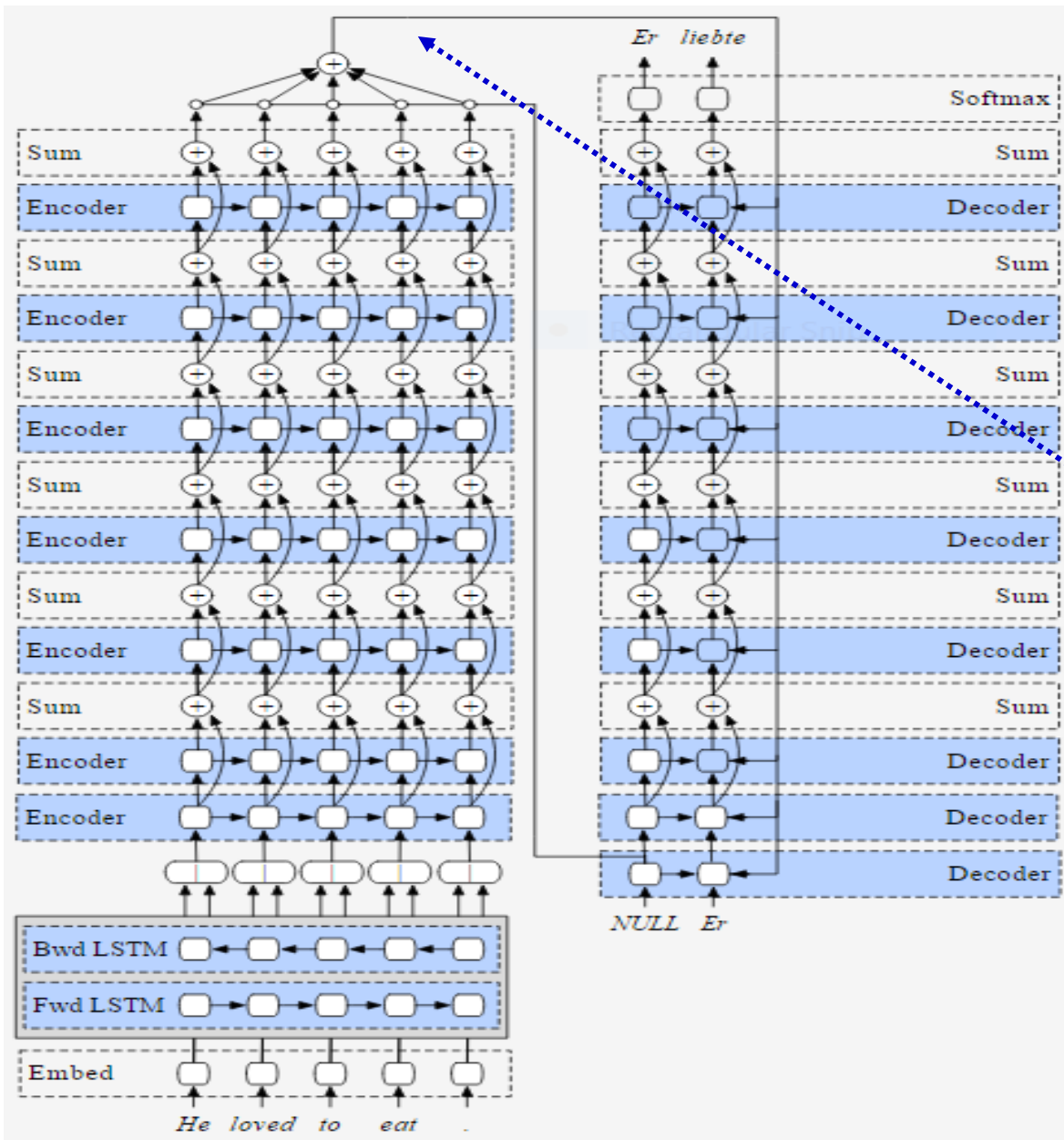
Dropout also applied to the output of the self-attention and feed forward sublayers, before adding the residual and normalizing.



Figures from J. Alammr's "The Illustrated Transformer"

(<https://jalammar.github.io/illustrated-transformer/>). Transformers paper: Vaswani et al.,
“Attention is All You Need”, 2017 (<https://arxiv.org/abs/1706.03762>).

Reminder: RNN-based MT system



Google's paper:
<https://arxiv.org/abs/1609.08144>

Images from Stephen Merity's
http://smerity.com/articles/2016/google_nmt_arch.html

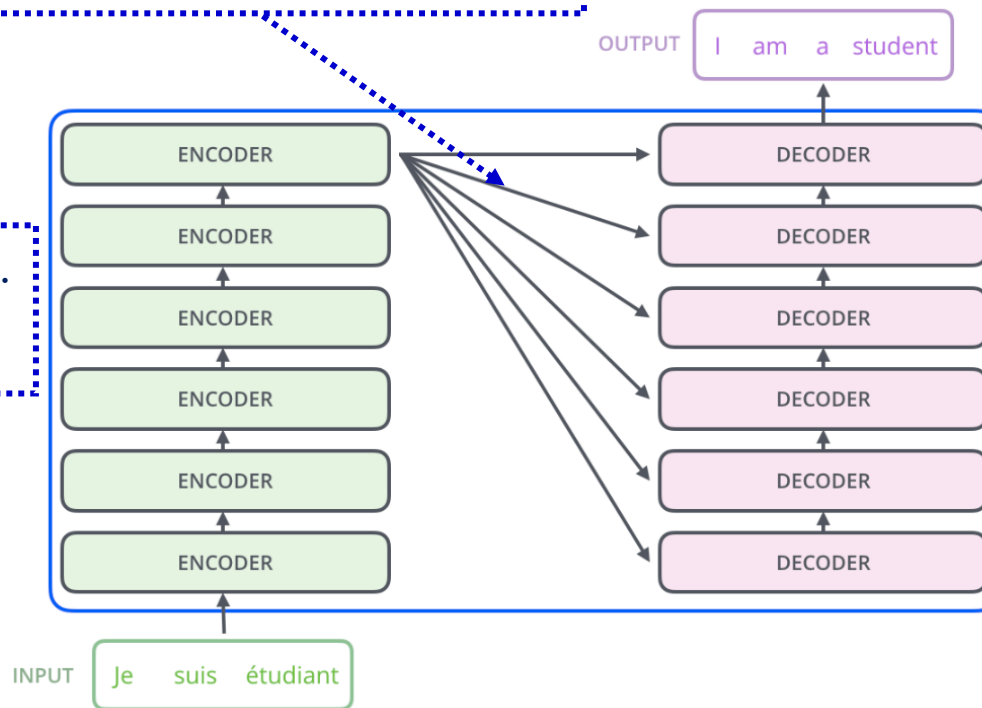
Attention over the states
of the encoder.

Stacked Transformer encoder-decoders

Attention over the vectors produced by the encoder. Keys (K) and Values (V) come from the vectors produced by the encoders. Queries (Q) come from the vectors of the decoder.

Optional
reading

Stacked encoders.
In BERT we use
only encoders.



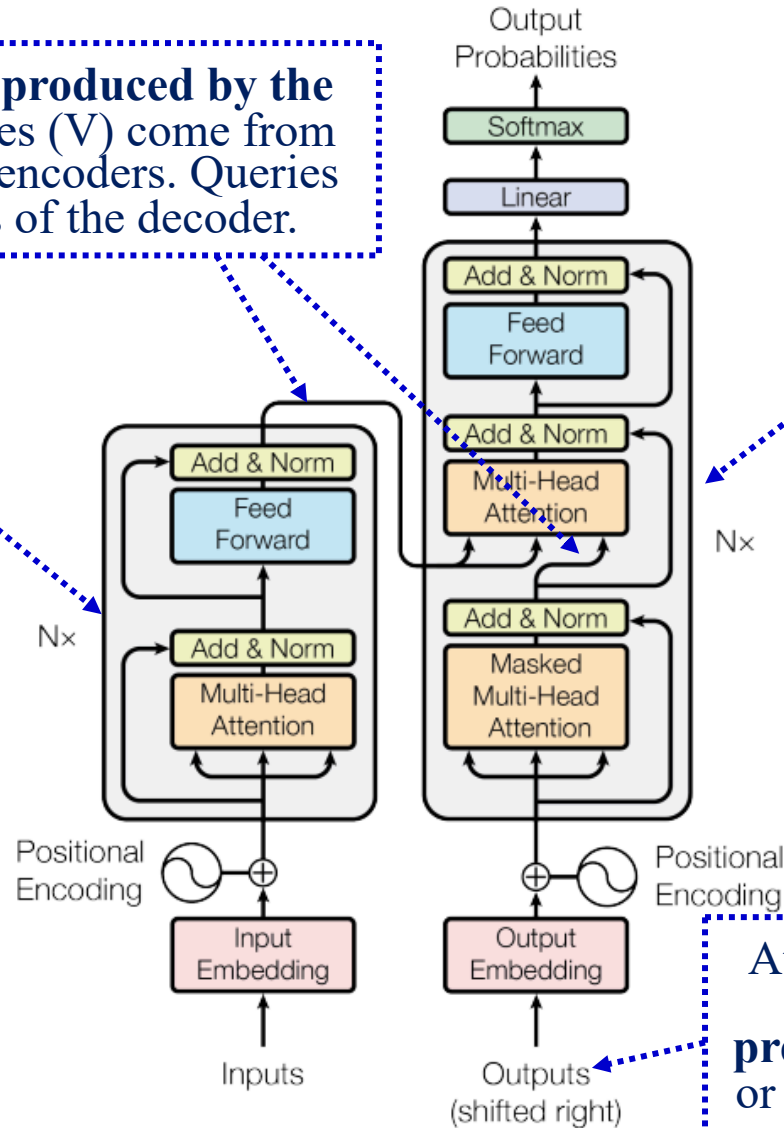
Stacked decoders.
For **machine translation**, we need both encoders and decoders. Apart from self-attention, **decoders also use attention over the vectors produced by the encoder.**

Figures from J. Alammam's "The Illustrated Transformer"
(<https://jalammar.github.io/illustrated-transformer/>). Transformers paper: Vaswani et al.,
"Attention is All You Need", 2017 (<https://arxiv.org/abs/1706.03762>).

Transformer-based Encoder-Decoder

Attention over the vectors produced by the encoder. Keys (K) and Values (V) come from the vectors produced by the encoders. Queries (Q) come from the vectors of the decoder.

N stacked encoders.
In **BERT** we use **only encoders**.



**Optional
reading**

N stacked decoders.
For **machine translation**, we need both encoders and decoders. Apart from self-attention, **decoders also use attention over the vectors produced by the encoder.**

At each word position, the **decoder sees only the preceding gold (at training) or generated (at test) words of the translation.**

Figure from Vaswani et al., “Attention is All You Need”, 2017.

<https://arxiv.org/abs/1706.03762>

BERT – Pretraining to predict masked words

Use the output of the masked word's position to predict the masked word

Possible classes:
All English words

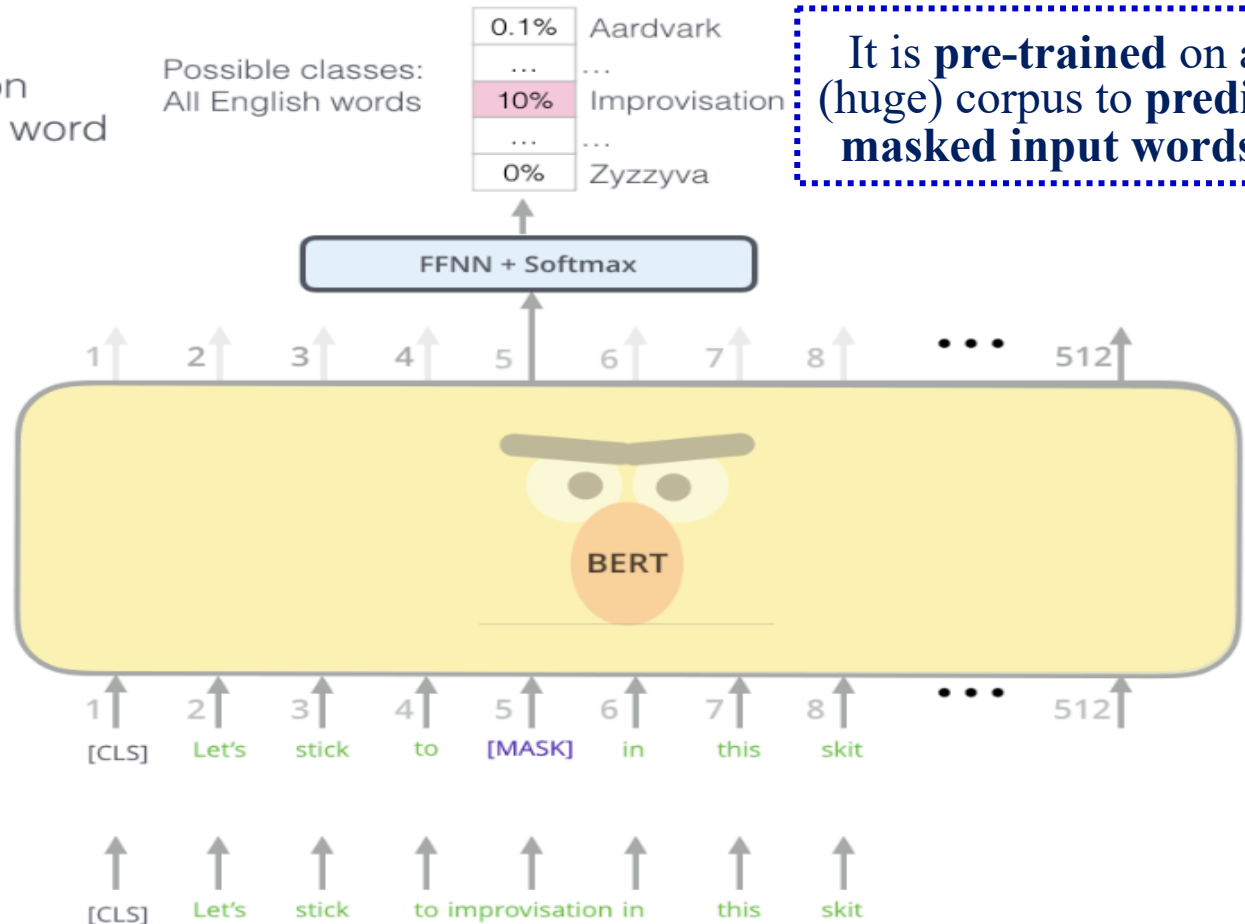
0.1%	Aardvark
...	...
10%	Improvisation
...	...
0%	Zyzzzva

It is **pre-trained** on a (huge) corpus to **predict masked input words**.

BERT uses **stacked Transformer encoders** (instead of RNNs or CNNs) to turn each **sequence of input embeddings** to a **sequence of context aware embeddings**.

Randomly mask 15% of tokens

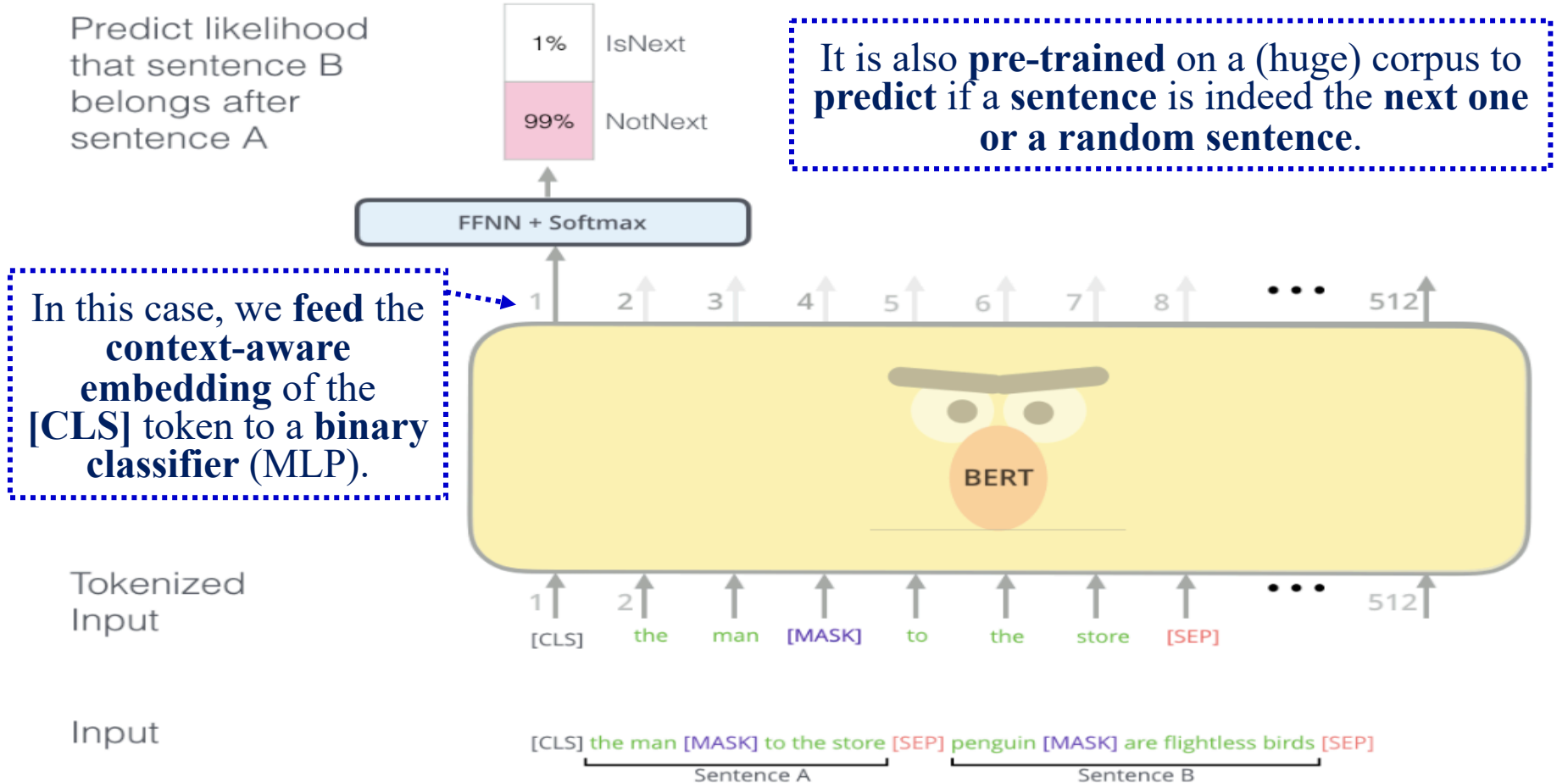
Input



BERT's clever language modeling task masks 15% of words in the input and asks the model to predict the missing word.

Figures from J. Alammari's "The Illustrated BERT, ELMo, and co."
(<http://jalammar.github.io/illustrated-bert/>). BERT paper: Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding", 2018
(<https://arxiv.org/abs/1810.04805>).

BERT – Pretraining to predict the next sentence



It is also **pre-trained** on a (huge) corpus to **predict** if a **sentence** is indeed the **next one** or a **random sentence**.

The second task BERT is pre-trained on is a two-sentence classification task. The tokenization is oversimplified in this graphic as BERT actually uses WordPieces as tokens rather than words --- so some words are broken down into smaller chunks.

Figures from J. Alammr's "The Illustrated BERT, ELMo, and co."
(<http://jalammar.github.io/illustrated-bert/>). BERT paper: Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding", 2018
(<https://arxiv.org/abs/1810.04805>).

BERT – Fine-tuning for sentence classification

We feed the **context-aware embedding** of the [CLS] token of each **sentence** to a **task-specific classifier** (e.g., MLP) that classifies the sentence (e.g., **Positive, Neutral, Negative** etc.)

Starting from the **pre-trained BERT**, we **jointly train BERT (further)** and the **task-specific classifier** on (possibly few) **task-specific training examples** (e.g., tweets + opinion labels).

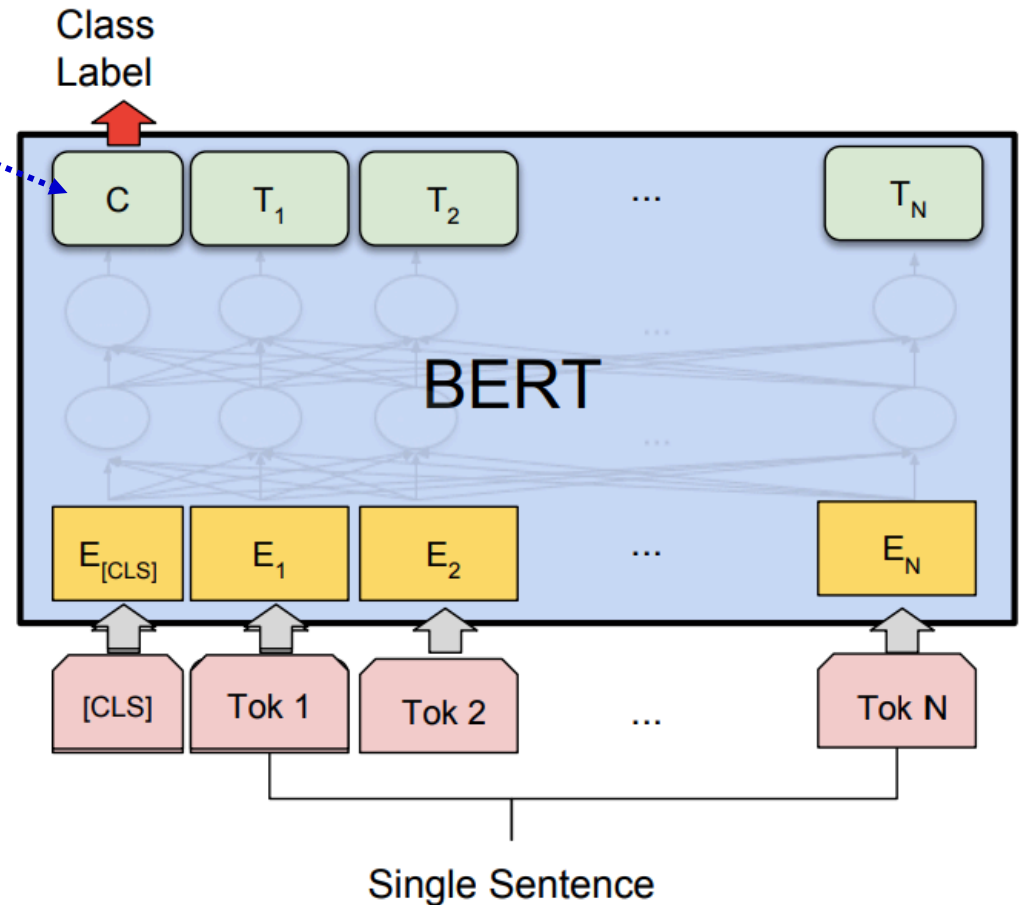


Figure from Devlin et al., “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”, 2018 (<https://arxiv.org/abs/1810.04805>).

BERT – Fine-tuning for sequence labeling

We feed the **context-aware embeddings** of the sentence's words to a **classifier** (e.g., MLP) that classifies them as **B-Per**, **I-Per**, **B-Org**, **I-Org**, ..., **Other**.

Starting from the **pre-trained BERT**, we **jointly train BERT (further)** and the **task-specific classifier** on (possibly few) **task-specific training examples** (manually labeled sentences).

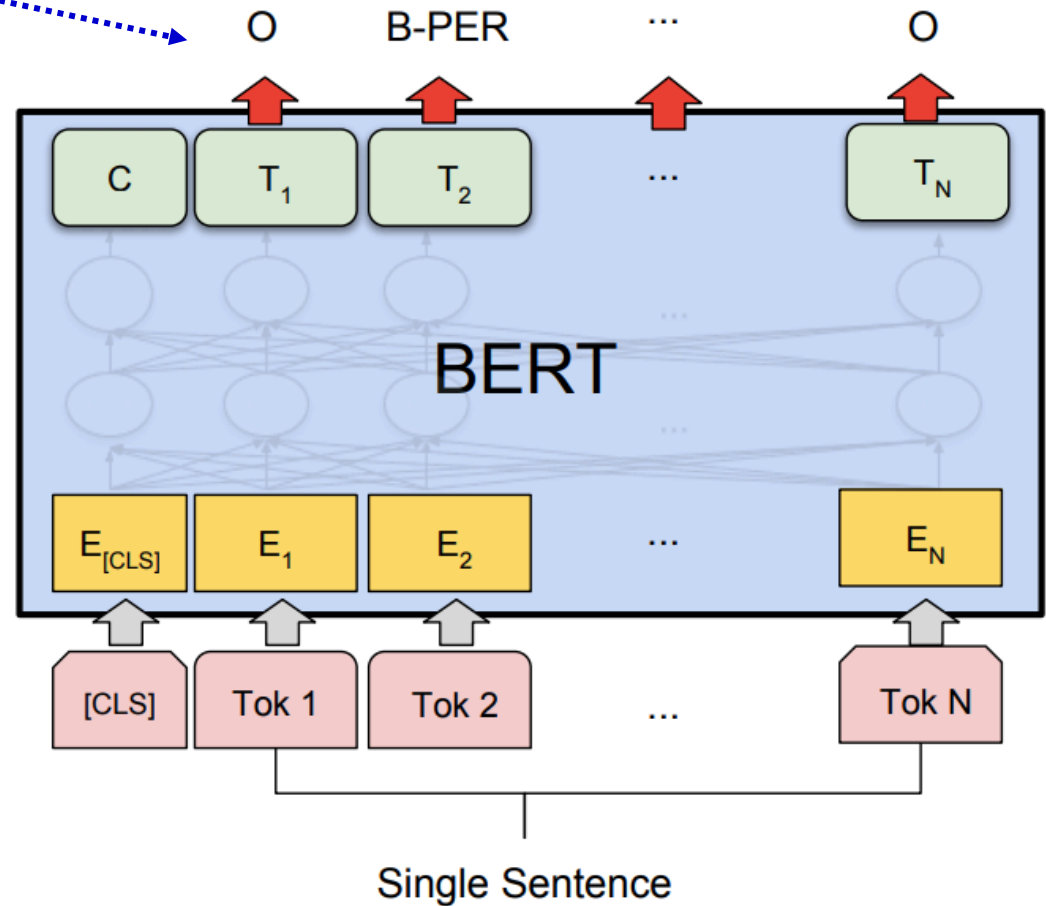


Figure from Devlin et al., “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”, 2018 (<https://arxiv.org/abs/1810.04805>).

BERT – Fine-tuning for textual entailment

We feed the **context-aware embedding** of the [CLS] token of each **sentence pair** to a **task-specific classifier** (e.g., MLP) that classifies the pair as **Entailment**, **Contradiction**, **Neutral**.

Starting from the **pre-trained BERT**, we **jointly train BERT (further)** and the **task-specific classifier** on (possibly few) **task-specific training examples** (annotated sentence pairs).

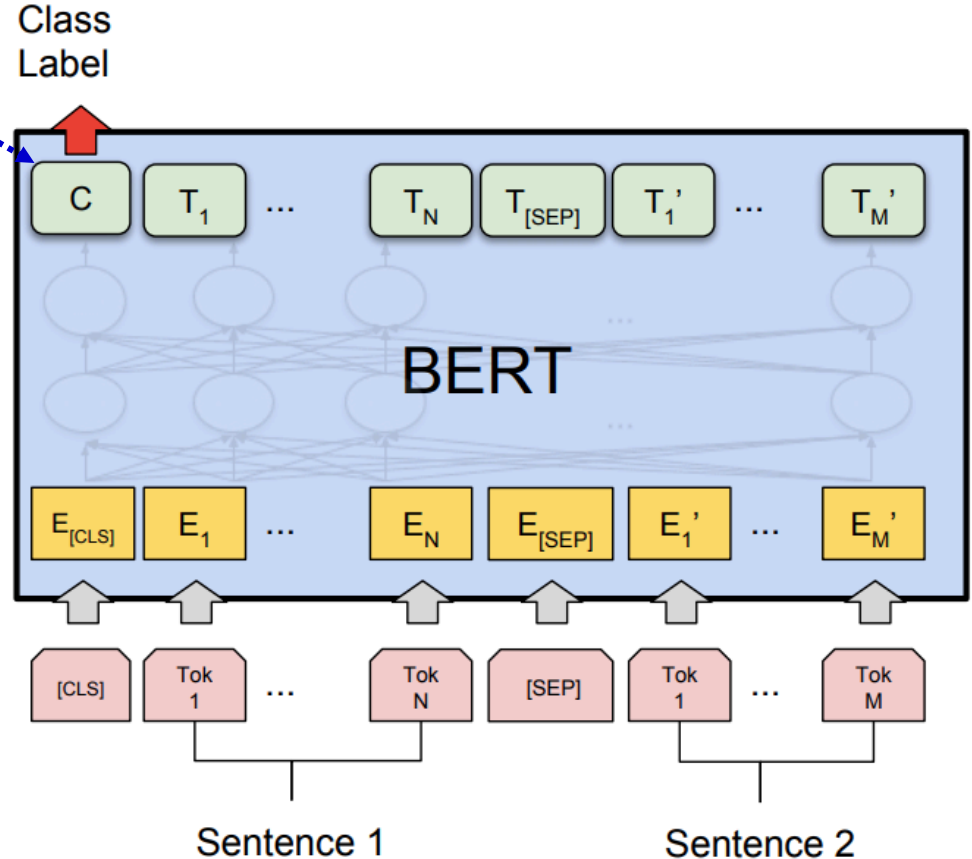


Figure from Devlin et al., “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”, 2018 (<https://arxiv.org/abs/1810.04805>).

Machine Reading Comprehension (MRC)

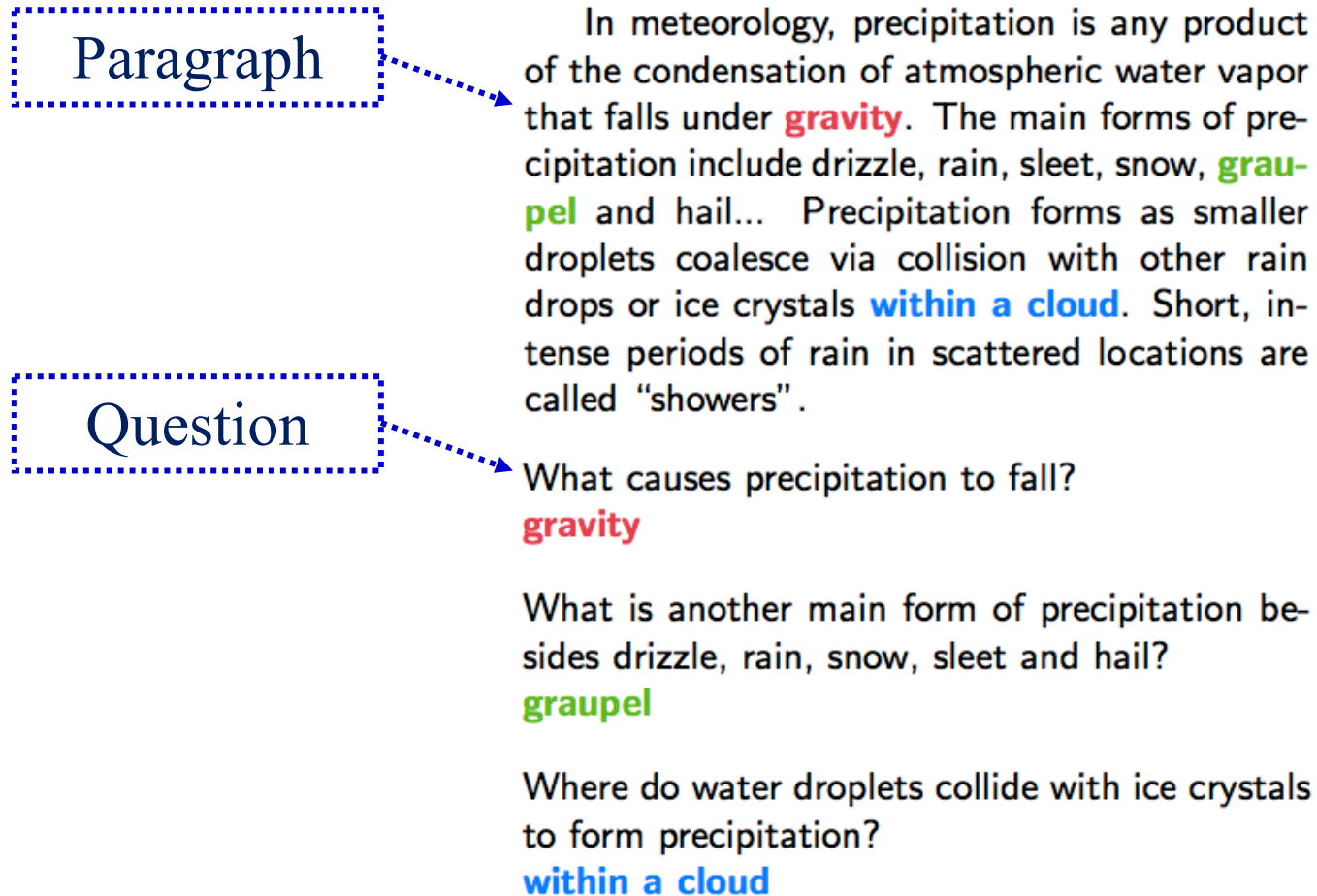


Figure from P. Rajpurkar et al., “SQuAD: 100,000+ Questions for Machine Comprehension of Text.”, EMNLP 2016 (<https://aclweb.org/anthology/D16-1264>).

BERT – Fine-tuning for MRC

We feed the **context-aware embeddings** of the paragraph's words to a **classifier** (e.g., MLP) that classifies them as **Start-Answer, End-Answer, Other**.

Starting from the **pre-trained BERT**, we **jointly train BERT (further)** and the **task-specific classifier** on (possibly few) **task-specific training examples** (paragraph-question pairs).

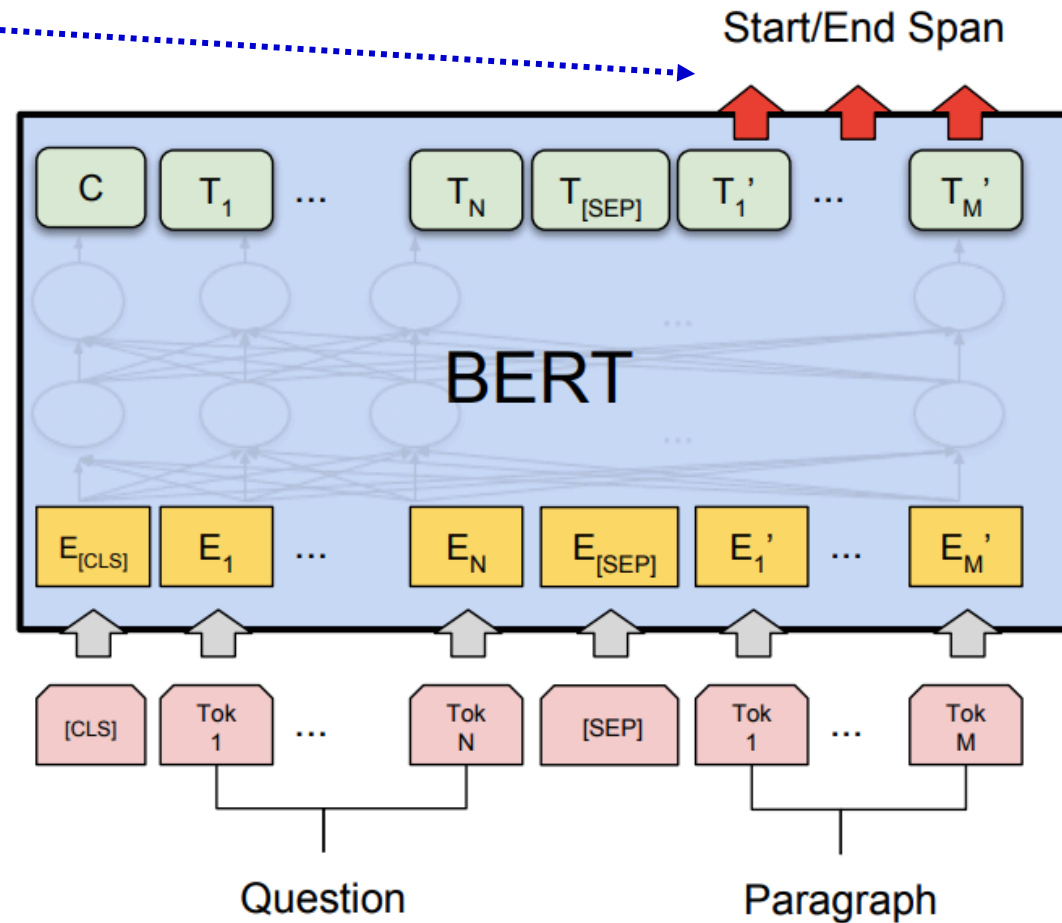


Figure from Devlin et al., “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”, 2018 (<https://arxiv.org/abs/1810.04805>).

Image captioning

Optional
reading

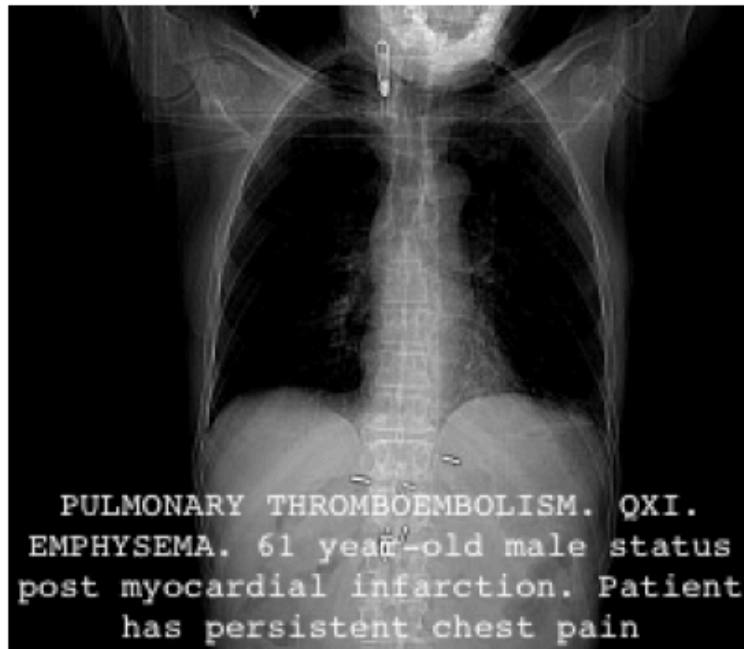


A blue and yellow train
traveling down train tracks.

(a) General

Possible applications:

- Image retrieval via captions.
- Eyesight problems.
- Drafting medical reports.



PULMONARY THROMBOEMBOLISM. QXI.
EMPHYSEMA. 61 year-old male status
post myocardial infarction. Patient
has persistent chest pain

(b) Biomedical

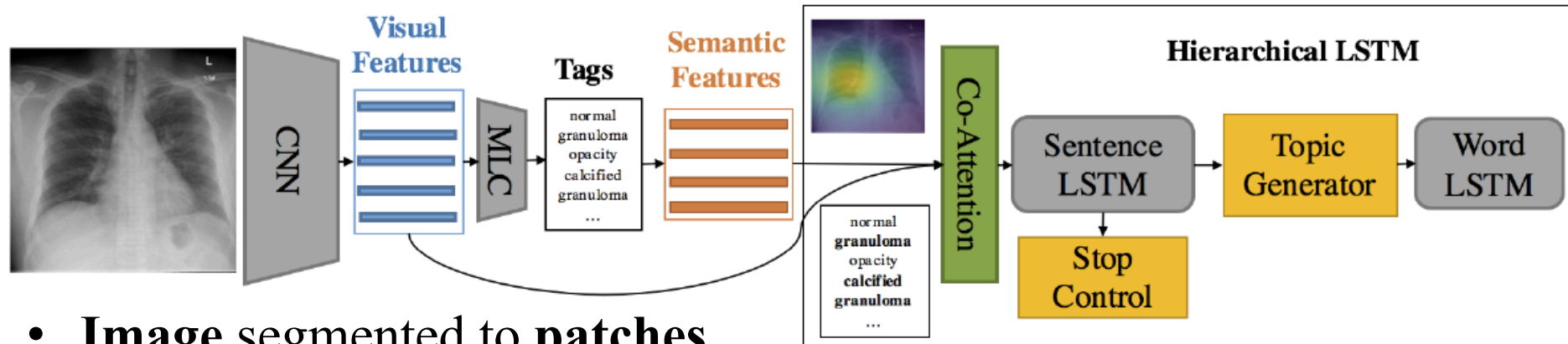
Figure 1: Example of a caption produced by the model of Vinyals et al. (2017) for a non-biomedical image (1a) and an example of a PEIR Radiology image with its associated caption (1b).

From I. Pavlopoulos, V. Kougia, I. Androutsopoulos, “A Survey on Biomedical Image Captioning”.

<https://www.aclweb.org/anthology/W19-1803/>

Biomedical image to text generation

Optional
reading

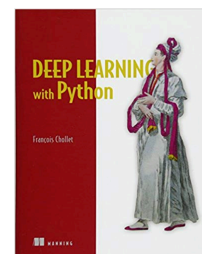
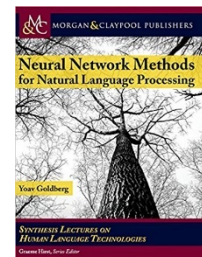


- Image segmented to **patches**.
- CNN converts each **patch** to a **vector**, producing “**visual features**”.
- MLP (“MLC”) predicts **tags** given the visual features.
- The **embeddings** of the tags are “**semantic features**”.
- **Sentence-level LSTM** produces **sentence embeddings** (“**topics**”).
 - A **stop control** (classifier) decides when to **stop producing sentences**.
 - At each time-step, **attention** over **visual** and **semantic features**.
- For each sentence embedding, **word-level LSTM** produces **words**.

B. Jing, P. Xie, E.P. Xing, “On the Automatic Generation of Medical Imaging Reports”,
ACL 2018 (<http://www.aclweb.org/anthology/P18-1240>).

Recommended reading

- Y. Goldberg, *Neural Network Models for Natural Language Processing*, Morgan & Claypool Publishers, 2017.
 - Mostly chapters 13.
- Jurafsky and Martin's, *Speech and Language Processing* is being revised (3rd edition) to include Deep Learning methods.
 - <http://web.stanford.edu/~jurafsky/slp3/>
- For an introduction to Keras (see labs) and practical DL for NLP and vision, see F. Chollet's *Deep Learning in Python*, Manning Publications, 2017.
- For an introduction to PyTorch and hands-on tutorials for NLP and vision, see the fast.ai free courses.
 - <https://course.fast.ai/>



Recommended reading

- Stanford's *NLP with Deep Learning* course.
 - Course material: <http://web.stanford.edu/class/cs224n/>
 - Videos: available on YouTube.
- CMU's *Neural Networks for NLP* course.
 - See: <http://www.phontron.com/class/nn4nlp2019/>
 - Videos: available on YouTube.
- For a detailed discussion of Transformers and a step-by-step PyTorch implementation, see “The Annotated Transformer” Jupyter notebook of A. Rush.
 - <http://nlp.seas.harvard.edu/2018/04/03/attention.html>
 - Also available as a Colab notebook (see links in notebook above).